



Junta de Andalucía
Consejería de Educación y Deporte

UNIDAD 10

PERSISTENCIA

—

Módulo de Programación



This work is licensed under a [Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-nc-sa/4.0/).

Autor: Fran Gómez

1. Bases de datos orientadas a objetos

- a. Características
- b. Diferencias con las bases de datos relacionales
- c. Instalación ObjectDB

2. JPA

- a. Tipos de datos
- b. Clases
 - i. Campos
 - ii. Métodos
- c. Mecanismos de consulta

3. Conexión

4. Operaciones CRUD

5. JPQL

- a. SELECT
- b. UPDATE
- c. DELETE

6. BBDD Relacionales

- a. JDBC
- b. CRUD
- c. Batch
- d. Metadatos
- e. Clientes gráficos

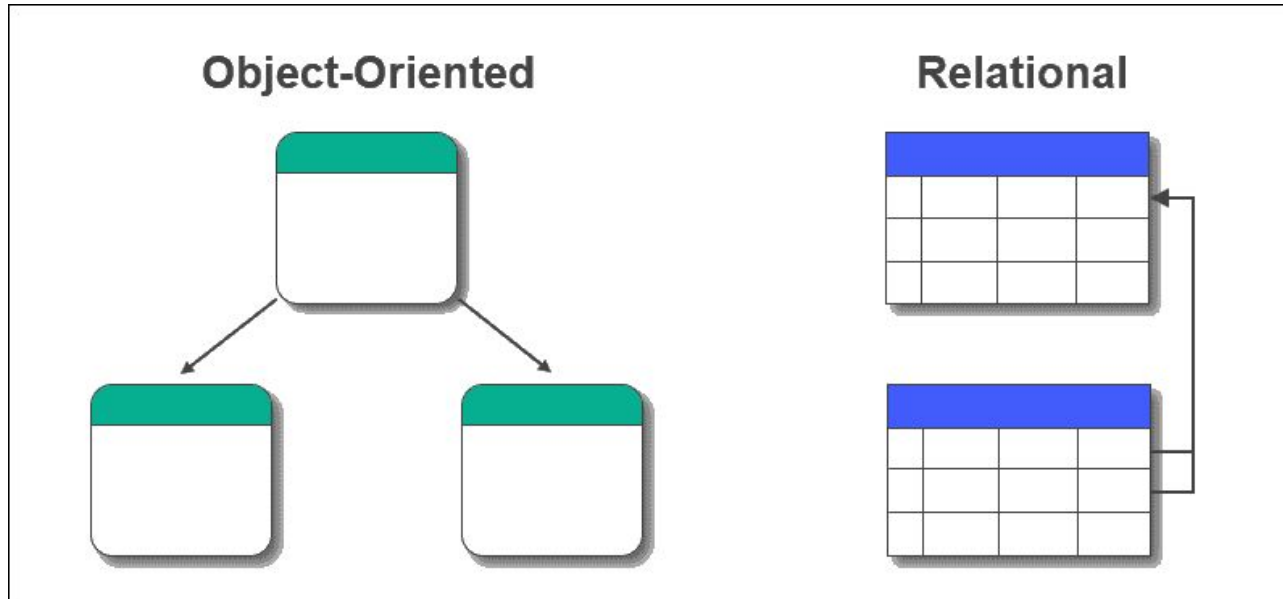
Criterios de Evaluación



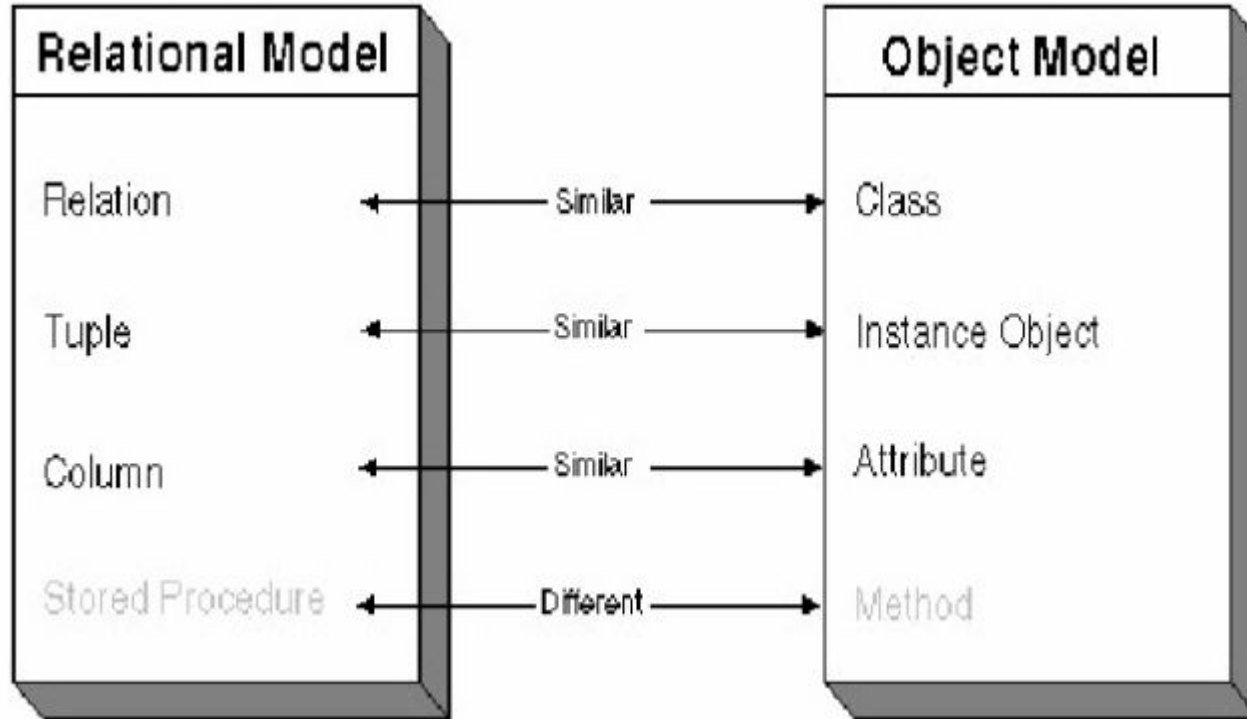
Resultado de Aprendizaje	% módulo	Criterio de Evaluación	% módulo	%RA
RA 8 Utiliza bases de datos orientadas a objetos, analizando sus características y aplicando técnicas para mantener la persistencia de la información.	10	a) Se han identificado las características de las bases de datos orientadas a objetos.	1	10
		b) Se ha analizado su aplicación en el desarrollo de aplicaciones mediante lenguajes orientados a objetos.	1	10
		c) Se han instalado sistemas gestores de bases de datos orientados a objetos.	1	10
		d) Se han clasificado y analizado los distintos métodos soportados por los sistemas gestores de bases de datos.	1	10
		e) Se han creado bases de datos y las estructuras necesarias para el almacenamiento de objetos.	1	10
		f) Se han programado aplicaciones que almacenen objetos en las bases de datos creadas.	1	10
		g) Se han realizado programas para recuperar, actualizar y eliminar objetos de las bases de datos.	2	20
		h) Se han realizado programas para almacenar y gestionar tipos de datos estructurados, como arrays, listas, etc.	2	20
RA 9 Gestiona información almacenada en bases de datos relacionales manteniendo la integridad y consistencia de los datos.	10	a) Se han identificado las características y métodos de acceso a sistemas gestores de bases de datos relacionales.	1	10
		b) Se han programado conexiones con bases de datos.	2	20
		c) Se ha escrito código para almacenar información en bases de datos.	1	10
		d) Se han creado programas para recuperar y mostrar información almacenada en bases de datos.	1	10
		e) Se han efectuado borrados y modificaciones sobre la información almacenada.	1	10
		f) Se han creado aplicaciones que ejecuten consultas sobre bases de datos.	2	20
		g) Se han creado aplicaciones para posibilitar la gestión de información presente en bases de datos.	2	20

- ❖ Persistir la información para usarla en distintas ejecuciones o transmitirla
- ❖ Ficheros vs BBDD
- ❖ OODB
- ❖ POJO (Plain Old Java Object) es una clase Java estándar que no depende de ningún framework, librería o interfaz especial / [JavaBeans](#)
- ❖ [Annotations](#)

- ❖ La información se representa mediante objetos



OODB vs RDB



ObjectDB es una base de datos orientada a objetos para Java.

- Usa las dos APIs estándar de Java: JPA y JDO
- No necesita realizar una correspondencia entre un modelo relacional y uno orientado a objetos, ya que es OO nativo.
- Instalación
 - Descarga del kit ObjectDB: <https://www.objectdb.com/download>
 - Importar la librería bin/objectdb.jar al proyecto del IDE
- Database Explorer → Herramienta GUI para realizar consultas, visualización y edición de contenido de base de datos: `java -jar explorer.jar`

Existen dos APIs nativas de Java:

- JPA (Java Persistence API)
- JDO (Java Data Objects)

Métodos de consulta:

- **JPQL** (JPA Query Language) → Se basa en la **sintaxis SQL**
- **JDOQL** (JDO Query Language) → Se basa en la sintaxis de Java
- JPA Criteria query → Construye consultas programáticamente sin escribir sentencias tipo SQL.

- Primitivos (int, char, float...)
- String
- Arrays (si su contenido son tipos persistentes o null)
- Envoltorio (Integer, Character, Float...)
- Tipos Math (BigInteger, BigDecimal)
- Tipos fecha y hora (Date, Time, Timestamp y Calendar)
 - java.sql
 - java.util → requieren anotación @Temporal

Crea un POJO que represente un evento musical. Deberá tener los campos para guardar el nombre del evento, la fecha y la recaudación. Establece el tipo que creas más adecuado para cada atributo.

Crea una clase de test con un main para crear una instancia de un evento como el anterior.

Responde a las siguientes cuestiones:

- ¿Cuándo conviene usar BigInteger en lugar de Integer?
- ¿Para qué sirve cada uno de los tipos: Date, Time, Timestamp y Calendar?

- Enumerados
 - Puede indicarse que se persista el valor numérico o alfanumérico →
`@Enumerated(EnumType.ORDINAL / STRING)`
- Tipos que implementan Serializable
- *Colecciones* o *Mapas* si contienen tipos persistentes o null
- Clases definidas por el usuario:
 - `@Entity`
 - ~~`@Mapped`~~
 - `@Embeddable`

- ❖ Representan una entidad (concepto similar al de tabla en BBDD) → @Entity
- ❖ De una clase sólo se almacenan los atributos (no los métodos) de tipos persistentes
- ❖ Además, la clase debe ser un POJO
 - Campos privados
 - Getter/Setters siguiendo Java Code Convention
 - Tener siempre un constructor sin parámetros
 - Serializable

```
@Entity (name = "Coche")  
public class Car implements Serializable {
```

@Embeddable → Este campo aunque sea de tipo una clase definida por el usuario, No crea una nueva entidad en la base de datos. Esto mejora el rendimiento, pero reduce su interoperabilidad.

```
@Entity (name = "cohecito")  
public class Coche implements Serializable {  
    no usages  
    @Embedded  
    private Fabricante fabricante;
```

```
@Embeddable  
public class Fabricante {  
    no usages  
    private String nombre;
```

Completa la clase del ejercicio anterior para que cumpla todas las restricciones de un POJO que podamos persistir en una BD, como acabamos de ver.

Además, añade un campo para almacenar el género musical dentro de un número determinado de géneros. Otro campo para llevar un control de la lista de artistas confirmados.

Cada artista no supondrá una nueva entidad en la capa de persistencia, aunque sí sea una clase más.

- ❖ Los campos de tipos persistentes se almacenarán en la bbdd

- ❖ Pero si no se desea que se almacenen:
 - @Transient
 - modificadores: final, static o transient

Transient

```
class Usuario implements Serializable {  
    private String nombre;  
    private transient String contraseña;
```



```
Usuario user = new Usuario("Fran", "secreta123");  
  
// Serializar el objeto  
ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("usuario.dat"));  
oos.writeObject(user);  
oos.close();  
  
// Deserializar el objeto  
ObjectInputStream ois = new ObjectInputStream(new FileInputStream("usuario.dat"));  
Usuario userLeido = (Usuario) ois.readObject();  
ois.close();  
  
System.out.println(userLeido);
```

```
Usuario: Fran, Contraseña: null
```

- ❖ Es conveniente siempre declarar una clave primaria: campo con valor único y no nulo para cada instancia (registro)
 - @Id
- ❖ Se puede indicar cómo se genera este ID
 - @GeneratedValue
 - Si se omite → el programa debe darle un valor al campo
 - strategy=GenerationType.AUTO → calcula el valor automáticamente
 - strategy=GenerationType.IDENTITY → autoincremento
 - strategy=GenerationType.SEQUENCE → secuencia personalizada
 - Requiere crear @SequenceGenerator
- ❖ Una clave primaria formada por varios campos → Crear clase @Embeddable



@Entity

```
public class Vehiculo implements Serializable {  
    @Id private Integer id;  
    private Integer numeroRuedas;
```

@Id

```
@GeneratedValue (strategy = GenerationType.SEQUENCE, generator = "referenciaSeq")  
@SequenceGenerator(initialValue = 50, allocationSize = 10, name = "referenciaSeq")  
private Integer referencia;
```

Ejercicio 3



Al ejercicio anterior, añade ahora un nuevo campo que se utilice como clave primaria. Su valor debe incrementarse de 10 en 10.

Por otro lado, introduce una variable de control llamada “control” de tipo int que no deberá reflejarse en base de datos, pues será utilizada sólo para la lógica del programa.

No se persisten.

- ❖ Una entidad que herede de otra, heredará sus campos persistentes

Ejemplo Herencia



```
public abstract class Persona {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    private String nombre;
```

```
@Entity  
public class Estudiante extends Persona {  
    private String curso;  
  
    // Getters y setters  
    public String getCurso() { return curso; }  
    public void setCurso(String curso) { this.curso = curso; }  
}
```

Tabla Estudiante

id	nombre	curso
1	Andrea	2º DAW
2	Marcos	1º ASIR

Recuerda: relación "tiene un" (has-a). → Una clase que tiene otra clase.

Existen dos formas de mapearla en BD con JPA:

- ❖ Clase embebida → `@Embedded` (solo crea la entidad principal)
- ❖ Relación → `@OneToOne`, `@ManyToOne`, etc. (crea entidades secundarias)

Ejemplo Composición embebida



```
@Embeddable
public class Direccion {
    private String calle;
    private String ciudad;
```

```
@Entity
public class Cliente {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;

    @Embedded
    private Direccion direccion;
```



Resultado en la tabla cliente

id	nombre	calle	ciudad
1	Laura	C/ Mayor, 5	Sevilla
2	Pablo	Av. Sol, 11	Cádiz

Ejemplo composición con relaciones



```
@Entity
public class DocumentoIdentidad {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String numero;
    private String tipo;
```

```
@Entity
public class Persona {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombre;

    @OneToOne(cascade = CascadeType.ALL)
    private DocumentoIdentidad documento;
```

Tabla documento_identidad

id	numero	tipo
1	12345678A	DNI

Tabla persona

id	nombre	documento_id
1	Ana López	1

❖ Objeto que representa la instancia de la bbdd → EntityManager

- Para obtenerlo abrimos una conexión de esta manera:

```
EntityManagerFactory emf = Persistence.createEntityManagerFactory("objectdb:$objectdb/db/coches.odb");  
EntityManager em = emf.createEntityManager();
```

- Después realizamos las operaciones (consultas CRUD)
- Finalmente cerramos la conexión así:

```
em.close();  
emf.close();
```

Las queries o consultas a bbdd pueden ser de 4 tipos, lo que se conoce como CRUD:

- Creation
- Read
- Update
- Delete

Toda operación que realice un cambio en la bbdd debe realizarse dentro de una transacción:

```
em.getTransaction().begin();  
ClaseEntidad objeto = new ClaseEntidad (arg1, arg2, ...);  
em.persist(objeto);  
em.getTransaction().commit();
```

Las distintas operaciones CRUD pueden realizarse mediante dos alternativas:

- Los métodos específicos de EntityManager
 - persist
 - find
 - remove
 - Para la modificación de un campo no hay que hacer nada más allá de cambiar el valor con el setter del POJO en cuestión.
- Consulta JPQL → Sentencia en una cadena de texto
 - ~~INSERT INTO~~
 - SELECT
 - DELETE
 - UPDATE
- Criterios

❖ Formas de declararlas

- Estática → En tiempo de compilación. Obliga uso de parámetros. Mayor eficiencia.

```
@NamedQuery(name="queryEstatica", query="SELECT ..."  
@Entity
```

```
Query query = em.createNamedQuery("queryEstatica");
```

- Dinámica → En tiempo de ejecución. No es inmutable, luego se pueden meter literales.

```
Query query = em.createQuery("SELECT ...");
```

❖ Formas de ejecutarlas

- Devuelve un único valor (máximo, media,...) o registro

```
ClaseEntidad objeto = query.getSingleResult();
```

- Devuelve varios registros

```
List<ClaseEntidad> lista = query.getResultList();
```

❖ Tipado

- Sin tipado → Clase Query (necesita casting)

```
Query query = em.createQuery("...");  
Clase objeto = (Clase)query.getSingleResult();
```

- Con tipado → Clase TypedQuery<Clase> (no necesita casting)

```
TypedQuery<Clase> query = em.createQuery("...", Clase.class);  
Clase objeto = query.getSingleResult();
```

- ❖ A tener en cuenta
 - No sensible mayúsculas/minúsculas salvo nombres
 - Cadena de texto con comillas simples
 - Booleanos se ponen como TRUE o FALSE

Inserción (Create)



Lo vamos ha hacer sólo
usando persist

```
// Abrir conexión
EntityManagerFactory emf = Persistence.
    createEntityManagerFactory( "objectdb:$objectdb/db/coches.oddb");
EntityManager em = emf.createEntityManager();

// Iniciar la transacción
em.getTransaction().begin();

// Operaciones
Coche c1 = new Coche();
c1.setColor("Rojo");

Coche c2 = new Coche();
c2.setColor("Verde");

em.persist(c1);
em.persist(c2);

// Finalizar la transacción: salvar la información
em.getTransaction().commit();

// Cerrar conexión
em.close();
emf.close();
```

Ejercicio 4



Crea una nueva base de datos llamada eventos.odb e inserta dos nuevos eventos en ella.

❖ Leer registros de la bbdd

```
ClaseEntidad objeto = em.find(ClaseEntidad.class, valorClavePrincipal);
```

- No hace falta Casting.
- Si no lo encuentra devuelve null.

```
Coche coche = em.find(Coche.class, o: 50);  
System.out.println("Coche: " + coche.getColor());
```

Recuperación (Read) usando JPQL



```
TypedQuery<Coche> query2 = em.createQuery( s: "SELECT c FROM cohecito c WHERE color LIKE ?1", Coche.class);  
query2.setParameter( i: 1, o: "Rojo");  
Coche coche2 = query2.getSingleResult();  
System.out.println(coche2);
```

Imprime el primer evento que insertaste en el ejercicio anterior utilizando el identificador.

Imprime el segundo evento utilizando JPQL. En este caso hazlo de dos formas:

- Con una query estática
- Con una query dinámica

Modificación (Update)



```
// Iniciar la transacción  
em.getTransaction().begin();  
  
// Obtener el coche rojo y cambiarle el color a rosa  
Coche cocheToUpdate = em.find(Coche.class, 0: 50);  
cocheToUpdate.setColor("Rosa");  
  
// Finalizar la transacción: salvar la información  
em.getTransaction().commit();
```

Modificación (Update) usando JPQL



```
// Iniciar la transacción
em.getTransaction().begin();

// Utilizando JPQL
Query updateQuery = em.createQuery("UPDATE cochecito SET color = 'Rosita' WHERE referencia = 50");
System.out.println("Registros actualizados: " + updateQuery.executeUpdate());

// Finalizar la transacción: salvar la información
em.getTransaction().commit();
```

Cambiale el nombre al primer evento para ponerlo en mayúsculas utilizando el identificador.

Cambia ahora el nombre al segundo evento para ponerlo también en mayúsculas, pero ahora utilizando JPQL. En este caso hazlo de dos formas nuevamente:

- Con una query estática
- Con una query dinámica

Borrado (Delete)



```
// Obtener el coche rojo y lo eliminamos  
em.getTransaction().begin();  
Coche cocheToDelete = em.find(Coche.class, 0: 51);  
em.remove(cocheToDelete);  
em.getTransaction().commit();
```

Borrado (Delete) usando JPQL



```
// Utilizando JPQL lo eliminamos
em.getTransaction().begin();
Query deleteQuery = em.createQuery("DELETE FROM cochecito WHERE referencia = ?1");
deleteQuery.setParameter(1, azulito.getReferencia());
System.out.println("Registros eliminados: " + deleteQuery.executeUpdate());
em.getTransaction().commit();
```

Elimina de la base de datos el primer evento, pero antes haz una copia con otro id diferente. Realiza el borrado utilizando el identificador.

Después elimina el segundo utilizando JPQL. En este caso hazlo de dos formas nuevamente:

- Con una query estática
- Con una query dinámica

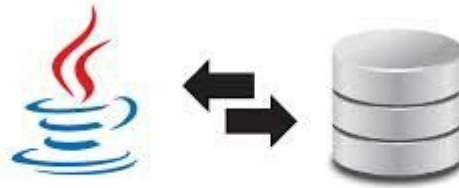
- Por posición

```
Query query3 = em.createQuery("SELECT c FROM cohecito c WHERE color LIKE ?1");  
query3.setParameter(1, "Rojo");
```

- Por nombre

```
TypedQuery query4 = em.createQuery("SELECT c FROM cohecito c " +  
    "WHERE color LIKE :color", Coche.class);  
query4.setParameter("color", "R%");
```

<https://www.objectdb.com/java/jpa/query/api>



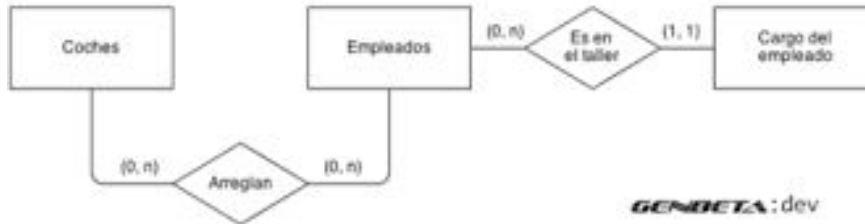
JAVA PERSISTENCE API

Objetivos:

- Instalar un SGBD relacional (MariaDB)
- Crear bbdd relacionales
- Aprender a conectarse a una bbdd mediante drivers
- Aplicar DML y DDL
- Navegar por los resultados de una query
- Utilizar asistentes para el manejo de bbdd relacionales

BDR $\Rightarrow$ Modelo Relacional e Integridad referencial

Modelo entidad-relación



FACTURAS	
FAC.NUM	CLIENTE
00001	1
00002	1
00003	2
00004	3

CLIENTES	
ID	NOMBRE
1	Nombre1
2	Nombre2
???	???

Arrows indicate referential integrity: three black arrows point from the first three rows of FACTURAS to the first two rows of CLIENTES. A red arrow points from the fourth row of FACTURAS to the third row of CLIENTES, which contains '???'.

Crear una BD mediante script



- Ejecutar cliente de línea de comandos.
- Crear script con: usuario, base de datos, tablas...

Se puede ejecutar el script completo o línea a línea en la consola.

```
MariaDB [tienda]> describe producto;
```

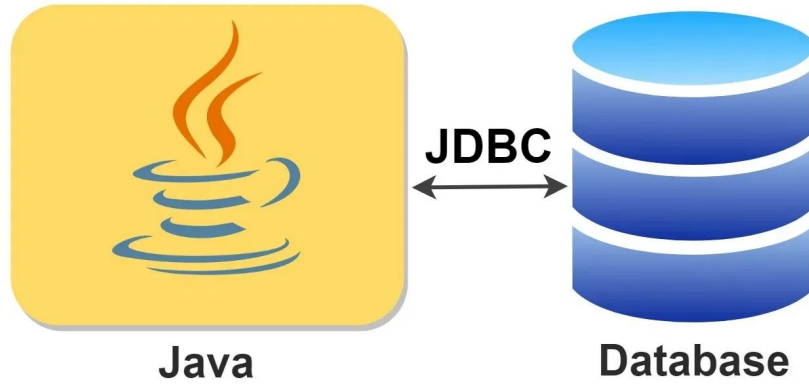
Field	Type	Null	Key	Default	Extra
id	varchar(10)	NO	PRI	NULL	
nombre	varchar(30)	YES		NULL	
descripcion	varchar(200)	YES		NULL	
precio	int(11)	YES		NULL	

```
4 rows in set (0.012 sec)
```

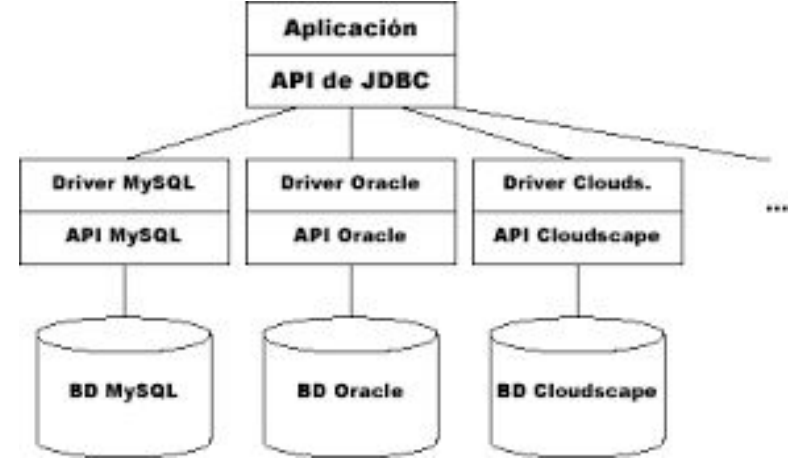
Java DataBase Connectivity



1º DAW
Programación
UD 10: BBDD



Fuente: Turing



Fuente: www.jtech.ua.es

Relación entre JDBC y JPA



JDBC → “hablar directamente en SQL con la base de datos”

JPA → “hablar en Java y que otro traduzca a SQL”

```
PreparedStatement ps = conn.prepareStatement("SELECT * FROM personas");  
ResultSet rs = ps.executeQuery();
```

```
Persona p = entityManager.find(Persona.class, 1);
```

Paquete java.sql [\[editar \]](#)

JDBC ofrece el paquete `java.sql`, en el que existen clases muy útiles para trabajar con [bases de datos](#).

Clase	Descripción
DriverManager	Para cargar un driver
Connection	Para establecer conexiones con las bases de datos
Statement	Para ejecutar sentencias SQL y enviarlas a las BBDD
PreparedStatement	La ruta de ejecución está predeterminada en el servidor de base de datos que le permite ser ejecutado varias veces
CallableStatement	Para ejecutar sentencias SQL de Procedimientos Almacenados.
ResultSet	Para almacenar el resultado de la consulta

Fuente: wikipedia

Conexión: ejemplo



```
try {
    Class.forName( className: "org.mariadb.jdbc.Driver"); // No necesario a partir de Java 6
    String url = "jdbc:mariadb://localhost:3306/tienda";
    Connection con = DriverManager.getConnection(url, user: "fran", password: "Fr4n");
    Statement stmt = con.createStatement();
    //TODO: Operaciones CRUD
    stmt.close();
    con.close();
} catch (SQLException e) {
    System.err.println("SQLException: " + e.getMessage());
} catch (ClassNotFoundException e) {
    System.err.println("No se encontró el driver: " + e.getMessage());
}
```

Ejercicio 8



Descarga el driver de MariaDB y construye un proyecto para conectar con la base de datos del ejercicio anterior. A continuación, carga el driver descargado en el proyecto e implementa un programa que se conecte con la base de datos y muestre el mensaje “Conexión exitosa” antes de cerrar la conexión.

Controlar también el caso en que la conexión no ha sido posible, mostrando el mensaje “No se pudo conectar” en su caso.

Statement: ejecutar sentencias



Statement stmt = con.createStatement();



```
String query = "SELECT * FROM producto WHERE pais = 'Espana'";

ResultSet resultados = stmt.executeQuery(query);

System.out.println("Listado de todos los productos españoles: ");

while (resultados.next()) {

    System.out.println("Producto " + resultados.getString("nombre"));

}
```

Método	Uso
executeQuery(sqlString)	Read
executeUpdate(sqlString)	Create, update, delete y DDL
execute(sqlString)	Cualquier comando SQL

ResultSet: manejar resultados



```
ResultSet result = stmt.executeQuery(query);
```

Apunta a antes del primer registro

Cursor

Objeto ResultSet

→	Nombre	Edad
→	Capi	25
→	Prats	31
→	Arzu	19

Apunta a después del último registro



`result.next();` → true

`result.next();` → true

`result.next();` → true

`result.next();` → false

next	Pasa a la siguiente fila
previous	Ídem fila anterior
last	Ídem última fila
first	Ídem primera fila
absolute(int fila)	Pasa a la fila número fila
relative(int fila)	Pasa a la fila número fila desde la actual
getRow	Devuelve la número de fila actual
isLast	Devuelve si la fila actual es la última
isFirst	Ídem la primera

ResultSet: obtener valores



resultados.**getXXXX**(fieldString)

getInt	Datos enteros
getDouble	Datos reales
getBoolean	Campos booleanos (si/no)
getString	Campos de texto
getDate	Tipo fecha (Devuelve Date)
getTime	Tipo hora (Devuelve Time)

```
int edad;  
String nombre;  
  
while(result.next()){  
    edad= result.getInt("Edad");  
    nombre = result.getString("Nombre");  
}
```

Ejercicio 9



Inserta 3 productos cualesquiera en la base de datos de tienda, utilizando el cliente de línea de comandos de MariaDB.

Muestra estos 3 productos por la consola ordenados por precio de mayor a menor.

stmt.executeUpdate(sqlQuery) → Devuelve el número de filas insertadas

```
String queryInsert = "INSERT INTO tienda.producto (id, nombre, descripcion, precio, pais) " +  
    "VALUES ('1', 'fresa', 'fresa portuguesa', 3, 'Portugal');";  
System.out.println("Se insertaron: " + stmt.executeUpdate(queryInsert)+ "filas");
```

```
String queryUpdate = "UPDATE tienda.producto SET precio = 6 WHERE id = '1';";  
System.out.println("Se actualizaron: " + stmt.executeUpdate(queryUpdate)+ "filas");
```

```
String queryDelete = "DELETE FROM tienda.producto WHERE id = '1';";  
System.out.println("Se eliminaron: " + stmt.executeUpdate(queryDelete)+ "filas");
```

El commit es automático. Para ponerlo manual: `con.setAutoCommit( false );`

Cada vez que se ejecuta una query (métodos execute de Statement), se realiza una transacción. → Muchas transacciones afectan negativamente el rendimiento.

Se pueden realizar varias queries en una única transacción:

```
// Desactivar auto-commit
con.setAutoCommit(false);
// Añadir queries al lote
stmt.addBatch(queryInsert);
stmt.addBatch(queryUpdate);
stmt.addBatch(queryDelete);
// Recuperar número de registros afectados
int[] numRegistrosAfectados = stmt.executeBatch();
// Confirmar cambios
con.commit();
```

Ejercicio 10



Escribe un programa que realice las siguientes operaciones en la bbdd tienda:

Borra todos los registros que tuviera la tabla productos.

Inserta 3 nuevos registros:

id	nombre	descripcion	precio	pais
1	manzana	Manzanas golden	50	Francia
2	pera	Peras conferencia	25	Espana
3	uva	Uvas groumet	30	Espana

Modifica el precio de la pera a 20 céntimos.

Ejercicio 11



- Investiga el uso del método `execute`.
- Imprime por consola todos los productos españoles usando este método.
- Ejecuta en un sólo lote todas las consultas anteriores.
- Ejecuta un nuevo batch con la inserción de estos nuevos

id	nombre	descripcion	precio	pais
4	kiwi	Zaspri gold	120	Nueva Zelanda
5	perito	Peritos de Tavizna	18	Espana
6	plátano	Plátano canario	40	Espana

Hay dos opciones:

- Hacer una query con el **Statement** (como hasta ahora)

```
String query = "SHOW DATABASES";  
ResultSet resultados = stm.executeQuery(query);  
System.out.println("Bases de datos existentes:");  
while (resultados.next()) {  
    System.out.println("- " + resultados.getString( columnIndex: 1));  
}
```

Bases de datos existentes:

- information_schema
- mysql
- performance_schema
- sys
- tienda

- Utilizar la clase **DatabaseMetaData**:

DatabaseMetaData

```
DatabaseMetaData metaData = con.getMetaData();
ResultSet resultados = metaData.getCatalogs();
System.out.println("Bases de datos existentes:");
while (resultados.next()) {
    System.out.println("- " + resultados.getString( columnLabel: "TABLE_CAT"));
}
```

Bases de datos existentes:

- information_schema
- mysql
- performance_schema
- sys
- tienda

Partiendo de la base de datos tienda, realiza una conexión y después imprime por consola los siguientes metadatos:

- Tablas de la base de datos tienda
- Columnas de la tabla producto
- Nombre del usuario que ha realizado la conexión
- URL de conexión
- Nombre y versión del driver utilizando, así como el nombre del SGBD.

Repositorio del profesor:

https://github.com/fgomrom/1daw-prog-22_23/tree/main/org/ieslosremedios/daw1/prog/ut8

Actividades finales



1º DAW
Programación

UD 10: BBDD

1. Práctica 8
2. Portfolio 8
- ~~3. Examen 8~~
4. Feedback 8